# Product & Pricing Service - Code Audit Report

**Generated: 2026-10-14 00:11:58**
**Service: Product & Pricing Microservice**
**Port: 8080**
**API Prefix: /api/v1/products**
**Technology Stack: Spring Boot 3.3.2, Java 21, Maven, MySQL, Kafka, Spring Security OAuth2**

---

## Executive Summary

The Product & Pricing Service acts as the central catalog and governance engine for banking products (FD, SAVINGS, LOAN). It encapsulates product identification, eligibility criteria, term ranges, interest rate bounds, fees, customer role assignments, and tiered rate variations. External modules consume product data and pricing logic exclusively via secure REST APIs.

## Architecture Overview

### Service Communication

Angular UI !' API Gateway !' Product Service (8080)
!“
FD Calculator Service (8084)
Customer Service (8082)

### Key Integrations

- **FD Calculator Service:** Provides product configuration via internal endpoint
- **Kafka:** Batch product creation logging and audit events
- **MySQL:** Product persistence with normalized schema

---

## API Endpoints

### Public Endpoints

---

## Configuration

### Environment Variables

- `DB_URL` - Database connection URL
- `DB_USER` - Database username
- `DB_PASSWORD` - Database password

- `JWT_SECRET` - JWT secret key (dev only)
- `JWT_ISSUER_URI` - JWT issuer URI (default: http://localhost:8081)
- `JWT_AUDIENCE` - JWT audience (default: product-service)
- `JWT_ROLES_CLAIM` - JWT roles claim (default: roles)
- `KAFKA_BOOTSTRAP_SERVERS` - Kafka bootstrap servers (default: localhost:9092)
- `SERVER_PORT` - Server port (default: 8080)
- `APP_CORS_ALLOWED_ORIGINS` - CORS allowed origins (default: http://localhost:4200,http://127.0.0.1:4200)

### Standardized Configuration

- Profile-based configuration (dev, prod)
- JPA ddl-auto: validate (prod), update (dev)
- Actuator endpoints: health, info, metrics, prometheus
- Structured JSON logging with correlation IDs
- Kafka configuration for batch processing
- CORS configuration (to be removed when API Gateway is implemented)

---

## Security

### Authentication

- OAuth2 Resource Server
- JWT validation
- HMAC-256 (dev) or JWKS (prod)

### Authorization

- Role-based access control via @PreAuthorize
- Roles: BANK_OFFICER, ADMIN, CUSTOMER
- Authorization constants in ProductAuthorization

### CORS

- Allowed origins: http://localhost:4200, http://127.0.0.1:4200
- To be removed when API Gateway is implemented

---

## Observability

### Structured Logging

- JSON structured logging via Logstash Encoder (logstash-logback-encoder 7.4)
- Correlation IDs in all logs
- Log files written to ./logs/product-service.log
- 30-day log retention
- Configuration: logback-spring.xml

## Metrics

- Actuator endpoints: /actuator/health, /actuator/metrics, /actuator/prometheus
- Custom business metrics (MetricsConfig):

- product.creation.total - Total products created
- product.update.total - Total products updated
- product.rate.calculations.total - Total rate calculations
- product.rate.calculations.duration - Rate calculation duration timer
- product.cache.hits - Cache hits
- product.cache.misses - Cache misses
- product.batch.operations.total - Total batch operations

## Kafka Integration

- Topics:

- product-service-batch-products - Batch product creation events
- product-service-audit-logs - Audit log events

- Environment variable: KAFKA_BOOTSTRAP_SERVERS (default: localhost:9092)

---

# Data Model

## Product Entity

- Table: product
- Primary Key: productId (UUID)
- Business Key: productCode (unique)
- Relationships:

- One-to-many: ProductBusinessRule
- One-to-many: ProductCharge
- One-to-many: ProductRole
- One-to-many: RateVariation (embeddable)

- Audit Columns: createdAt, updatedAt, createdBy, updatedBy
- Uses @PrePersist and @PreUpdate hooks

## Related Entities

- ProductBusinessRule - Term and rate limits
- ProductCharge - Fees and charges
- ProductRole - Allowed customer roles
- RateVariation - Rate variations by category

---

# Conventions

## Naming Conventions

- Java fields: camelCase
- Database columns: snake_case
- API endpoints: kebab-case
- DTOs: Lombok @Data, @Builder for mutable DTOs, Java Records for immutable DTOs
- Services: Interface + Impl pattern
- Exceptions: Descriptive names ending with "Exception"

## Code Conventions

- Use Lombok for DTOs and Entities (builder pattern)
- Use Java Records for immutable DTOs (RateCalculationResponseDTO, TermLimitDTO, RateLimitDTO)
- Use @Valid for request validation
- Use @NotNull, @NotBlank, @DecimalMin, @DecimalMax, @Pattern for field validation
- Use @Operation and @ApiResponse for Swagger documentation
- Use @SecurityRequirement for endpoint security
- Use @PreAuthorize for role-based authorization

## Monetary Values

- All monetary values use BigDecimal
- Currency amounts: precision 18, scale 2
- Rates: precision 6, scale 2
- Rounding mode: HALF_UP

---

# Key Components

## Controllers

- ProductController - REST API endpoints

## Services

- ProductService - Main business logic
- ProductServiceImpl - Service implementation

## Repositories

- ProductRepository - JPA repository
- ProductSpecifications - JPA specifications for dynamic queries

## Mappers

- ProductMapper - Entity-DTO mapping

## DTOs

- ProductRequestDTO, ProductResponseDTO (Lombok)
- RateCalculationResponseDTO (Java Record)

- TermLimitDTO, RateLimitDTO (Java Records)
- ChargesDTO, RateVariationDTO

### Entities

- Product, ProductBusinessRule, ProductCharge, ProductRole
- RateVariation (embeddable)

### Enums

- ProductStatus: ACTIVE, INACTIVE, DRAFT, SUSPENDED, RETIRED
- ChargeType: interestCredit, tax, withdrawalFee

---

## Dependencies

### Core Spring

- Spring Boot Starter Web
- Spring Boot Starter Data JPA
- Spring Boot Starter Validation
- Spring Boot Starter Security
- Spring Boot Starter OAuth2 Resource Server
- Spring Boot Starter Actuator
- Spring Boot Starter Cache

### Database

- MySQL Connector
- H2 Database (dev/testing)

### External Services

- Spring Boot Starter Kafka

### Documentation

- Springdoc OpenAPI Starter WebMVC UI

### Utilities

- Lombok
- JJWT

### Observability

- Micrometer Core
- Logstash Logback Encoder 7.4

---

## Deployment

### Docker

- Base image: eclipse-temurin:21-jre-alpine
- Non-root user: appuser
- Container support enabled
- MaxRAMPercentage: 75%
- Port: 8080

### Build

mvn clean package

### Run

docker build -t product-service .
docker run -p 8080:8080 product-service

---

## Testing

### Test Coverage

- ProductEndToEndIntegrationTest - Full context with real JWT
- ProductSecurityTest - Security filters and role authorization
- ProductControllerTest - HTTP contracts and validation
- ProductServiceTest - Business rules and calculations
- FdCalculatorServiceTest - FD calculation regression
- OpenApiExportTest - OpenAPI spec export

### Running Tests

mvn test

---

## Integration Points

### FD Calculator Service

- Endpoint: /internal/products/{productCode}/calculator-config
- Purpose: Provide product configuration for FD calculations
- Authentication: JWT required
- Response: CalculatorConfigDTO

### Kafka

- Topics: product-service-batch-products, product-service-audit-logs
- Purpose: Batch processing and logging

---

# Recent Updates

## Standardization Improvements

1. **Monetary Values** - All monetary values standardized to BigDecimal
2. **Response Formats** - RateCalculationResponseDTO converted to Java record
3. **Environment Variables** - Standardized to DB_URL, DB_USER, DB_PASSWORD, JWT_SECRET
4. **Configuration Structure** - Reorganized to match FD Calculator
5. **Actuator Configuration** - Added Prometheus metrics export
6. **Structured Logging** - Added JSON logging with Logstash Encoder, correlation IDs
7. **Metrics** - Added custom business metrics via MetricsConfig
8. **Audit Entity** - Standardized @PrePersist/@PreUpdate hooks
9. **Docker** - Created Dockerfile matching FD Calculator structure
10. **API Documentation** - Standardized security scheme name to bearerAuth

---

# Compliance

## Banking Standards

- 11-column audit trail (created_at, updated_at, uuid, ooo_user_id, ooo_ws_id, ldbid, ooo_acpt_ts, ooo_host_ts, ooo_local_ts, ooo_acpt_ts_utc_ofst, rule_system_id)
- UUID for persistent identification
- CreatedBy/UpdatedBy tracking
- Automatic audit field population

## Security Standards

- JWT-based authentication
- Role-based authorization (ADMIN, BANK_OFFICER)
- Stateless session management
- CORS configuration (dev only)

---

# Future Enhancements

- Product versioning
- Product approval workflow
- Advanced search with full-text
- Product templates
- Bulk import/export
- Product lifecycle management
- Rate history tracking
- Dynamic rate calculation rules

---

# Documentation

- Swagger UI: http://localhost:8080/swagger-ui/index.html
- OpenAPI Spec: http://localhost:8080/v3/api-docs
- Health: http://localhost:8080/actuator/health
- Metrics: http://localhost:8080/actuator/metrics
- Prometheus: http://localhost:8080/actuator/prometheus

---

# Contact

**Team: Banking Technology Lab – Product Team**
**Email: product-team@bank.internal**
**Repository: Product & Pricing Service**

---

*End of Code Audit Report*